

몬스터(Mob) 오브젝트 풀링 아키텍처 분석 보고서

Unreal Engine 5.8 기반 FC 프로젝트 시스템 분석 및 타당성 검토

Project: FC (Lead Architecture)
Date: 2026-09-08
Status: Completed (Architectural Decision)
Author: Lead Systems & Multiplayer Architect

1. 핵심 요약 (Executive Summary)

⚠️ 최종 아키텍처 결론: 몹(ACharacter) 오브젝트 풀링 도입 비권장 (Reject)

현재 FC 프로젝트의 전투 스케일(캠프당 3~5마리 몬스터 관리)에서 `ACharacter` 기반 몬스터에 오브젝트 풀링을 적용하는 것은 **성능적 실익(Gain)**이 극히 미미한 반면, **상태 오염(Dirty State)** 및 **네트워크 리플리케이션 결합 위험(Risk)**은 **기하급수적으로 증가**합니다. 따라서 현재는 언리얼 엔진 네이티브 라이프사이클(`SpawnActor` / `Destroy`)을 유지하고, 풀링 최적화는 **발사체(Projectile)** 및 **시각 이펙트(VFX)**에 한정하여 적용하는 것을 강력히 권고합니다.

2. 분석 배경 및 풀링의 목적

일반적인 게임 엔진 개발 환경(Unity 등)에서 오브젝트 풀링(Object Pooling)은 오브젝트의 생성과 파괴가 빈번할 때 힙 메모리 할당/해제 비용을 절감하고 가비지 컬렉션(GC) 스파이크를 방지하기 위해 널리 사용되는 최적화 기법입니다. 본 분석은 FC 프로젝트 내 `AFCMobCharacter` 및 `AFCMobSpawnerBase` / `AFCMobCampSpawner` 환경에서 몬스터 풀링 도입이 가져올 실질적인 효과와 기술적 리스크를 종합적으로 평가하기 위해 수행되었습니다.

3. UE5 Actor 풀링의 본질적 기술 난점 (Hard Challenges)

유니티의 단순 컴포넌트 풀링과 달리, 언리얼 엔진 5에서 **ACharacter + AIController + GAS + Network Replication**이 결합된 복합 액터를 풀링하는 것은 엔진의 기본 아키텍처와 충돌하며 심각한 기술 부채를 유발합니다.

3.1 액터 표준 라이프사이클(Lifecycle)의 우회

- 언리얼 엔진의 핵심 서브시스템(NavMesh, World Partition, AI Perception, Subsystems 등)은 `BeginPlay()`, `EndPlay()`, `Destroyed()` 이벤트를 기준으로 액터를 등록하고 해제합니다.
- 풀링을 위해 액터를 파괴하지 않고 `SetActorHiddenInGame(true)` 및 `SetActorEnableCollision(false)` 상태로 유지하면, 비활성 몹이 월드 파티션 셀이나 네비게이션 공간 인덱스에 잔류하여 불필요한 인덱싱 연산을 유발하거나 타겟팅 시스템에 유령 객체로 남는 부작용이 발생합니다.

3.2 네트워크 리플리케이션 및 넷 도먼시(Net Dormancy)의 복잡도

FC 프로젝트의 핵심 원칙은 **"Single-Player is Local Multiplayer (Listen Server Replication)"**입니다. 풀링된 액터는 네트워크 상에서 다음과 같은 심각한 동기화 문제를 야기합니다:

- 대역폭 낭비 및 Dormancy 관리:** 풀에 반환된 액터의 속성이 클라이언트로 계속 복제되지 않도록 `DORM_DormantAll` 로 휴면시키고, 재스폰 시 `FlushNetDormancy()` 를 명시적으로 호출해야 합니다.
- 클라이언트 아티팩트:** 서버에서 위치를 이동하고 상태를 리셋할 때 패킷 손실이나 지연이 발생하면 클라이언트 뷰포트에서 순간적인 텔레포트 잔상, 굳은 자세(T-Pose), 바닥 관통 등의 글리치가 빈번하게 발생합니다.

- **RepNotify 충돌:** `bIsDead` 가 false로 전환되는 시점에 클라이언트에서 메시 콜리전, 모션 블렌딩, 머티리얼 파라미터 복원이 정확히 역순으로 초기화되어야 합니다.

3.3 Gameplay Ability System (GAS)의 상태 오염 (Dirty State)

`AFCMobCharacter` 는 `UAbilitySystemComponent` (ASC)와 `UFCAAttributeSet` 을 소유하고 있습니다. 풀에 반환하기 위해서는 다음 항목을 단 하나도 빠짐없이 **완전 초기화(Wipe)**해야 합니다:

```
// 풀 반환 시 요구되는 GAS 초기화 체크리스트
1. AbilitySystemComponent->CancelAllAbilities();
2. AbilitySystemComponent->RemoveActiveGameplayEffectBySource(...); // 모든 버프/DoT 제거
3. AbilitySystemComponent->ClearAllGameplayTags(); // 잔류 태그 초기화
4. AttributeSet->SetHealth(AttributeSet->GetMaxHealth()); // 체력 및 스탯 복원
5. PredictionKey, TargetData, GameplayCue 지속성 정리
```

💣 오염 위험: 단 하나의 DoT 또는 태그 누수

만약 이전 생명주기의 화염 지속 피해(DoT Effect)나 스톤 상태 태그가 1개라도 남아있는 상태로 풀에서 재배치될 경우, 새로 등장한 몬스터가 등장 즉시 사망하거나 스킬을 시전하지 못하는 치명적인 버그가 발생합니다.

3.4 AIController / BrainComponent / Blackboard 상태 관리

AFCMobAIController 는 비헤이비어 트리(BehaviorTree)와 블랙보드(Blackboard), AI 감각(Perception)을 구동합니다:

- 풀 반환 시 BrainComponent->StopLogic("Pooled") 및 진행 중인 모든 MoveTo 경로 탐색을 취소해야 합니다.
- 재활성화 시 HomeLocation , TargetActor 등 블랙보드의 모든 키를 수동으로 클리어하고 RestartLogic() 을 호출해야 합니다.
- AI Perception Component가 이전 타겟에 대해 보유하던 시각/청각 자극(Stimuli) 캐시가 남아있으면 비활성화 중에도 플레이어를 계속 추적하려는 문제가 발생합니다.

3.5 래그돌(Ragdoll) 물리 및 스켈레탈 메시 복구 비용

몬스터 사망 시 물리 시뮬레이션(Ragdoll)을 활성화하거나 사망 몽타주를 재생합니다:

- 풀에서 재사용할 때 Mesh->SetSimulatePhysics(false) 를 호출하고, 스켈레탈 메시 트랜스폼을 캡슐 컴포넌트의 루트 본에 다시 정밀하게 결합(AttachToComponent)해야 합니다.
- 애니메이션 인스턴스(AnimInstance)의 상태 머신 노드가 초기화되지 않으면 몬스터가 바닥에 누운 자세 그대로 슬라이딩하며 이동하는 비주얼 결함이 발생합니다.

4. FC 프로젝트 환경 진단 및 스폰 비용 분석

4.1 현재 몬스터 스폰너 사양

- 대상 클래스: AFCMobCampSpawner (캠프 기반 리스폰 컨트롤러)
- 동시 개체 수: 캠프당 MinMobCount = 3 ~ MaxMobCount = 5 마리 유지
- 스폰 주기: 결손 보충 시 5.0f ~ 10.0f 초 딜레이, 전멸 후 리스폰 시 10.0f ~ 20.0f 초 딜레이
- 소멸 정책: 사망 연출 후 DeathDespawnDelay = 5.0f 초 뒤 네이티브 Destroy() 호출

4.2 비용 대 효과 (Cost vs Benefit) 분석

구분	네이티브 Spawn/Destroy (현재 방식)	액터 오브젝트 풀링 (Object Pooling)
스폰 프레임 타임	약 0.15ms ~ 0.3ms (스폰 시 1회)	약 0.05ms (트랜스폼 및 가시성 변경)
평균 호출 빈도	5~10초에 단 1회	5~10초에 단 1회
메모리 상주 비용	최적 (필요한 수량만 할당)	높음 (미리 풀링된 Mesh/ASC/AI 상시 점유)
구현 복잡도	낮음 (엔진 표준 API)	극도로 높음 (수백 줄의 상태 리셋 코드)
버그 발생 위험도	0% (항상 깨끗한 인스턴스 생성)	매우 높음 (GAS, 래그돌, AI 상태 누수)

💡 성능 진단 결과: 측정 가능한 수준의 Hitch 부재

현재 스폰 빈도(5~10초에 1회)에서는 SpawnActor 가 유발하는 프레임 비용이 전체 프레임 타임(16.6ms / 60FPS 기준)의 1~2% 미만으로 엔진 프로파일러에서 스파이크로 감지조차 되지 않는 수준입니다. 풀링을 통해 얻을 수 있는 0.1ms의 이득에 비해 감수해야 할 시스템적 리스크가 압도적으로 큼니다.

5. 대체 권장 아키텍처 및 최적화 로드맵

🎯 권장 최적화 1: 발사체(Projectile) 및 VFX 풀링 적용 (High ROI)

몬스터와 달리 `AFCProjectileBase` 및 `AFCFireballProjectile`은 `AIController`나 복잡한 애니메이션/래그돌이 없고 상태가 단순합니다. 동시에 전투 중 초당 수십 개가 생성/소멸되므로 발사체 풀링은 구현 복잡도가 매우 낮으면서도 실제 GC 및 성능 향상 효과가 매우 뛰어납니다.

권장 최적화 2: 대규모 스폰 시 타임 슬라이싱 (Time-Slicing) 분산 스폰

만약 던전 입장이나 특정 웨이브 트리거로 30마리 이상의 몬스터가 한꺼번에 생성되어야 하는 상황이 발생할 경우, 풀링 대신 시간 분할 스폰(Time-Slicing)을 적용합니다.

- 단일 틱에 30마리를 일괄 `SpawnActor` 하지 않고, 타이머 또는 태스크 큐를 통해 `0.05초` 간격으로 1~2마리씩 분산 스폰합니다.
- 풀링의 복잡도를 일절 유발하지 않고도 스폰 히치를 완벽하게 방지할 수 있습니다.

권장 최적화 3: 대규모 군집(1,000+ 개체) 확장 시 UE5 Mass AI 도입

향후 핵앤슬래시나 대규모 좀비 웨이브 장르처럼 수백~수천 마리의 군집이 필요해질 경우, 언리얼 엔진 5의 공식 ECS 프레임워크인 **Mass Entity / Mass AI**를 활용해야 합니다.

- 원거리 개체는 초경량 Mass Entity(UObject가 아닌 C++ 구조체 기반)로 가볍게 시뮬레이션합니다.
- 플레이어와 교전하는 근거리 영역에 진입했을 때만 `MassRepresentationProcessor`를 통해 `ACharacter`로 전환 (LOD Swap)하는 것이 언리얼 엔진 5.8의 표준 고성능 솔루션입니다.

6. 최종 결론 및 제언

1. **현 단계 도입 거부:** FC 프로젝트의 현 구조와 기획 스케일 상 몬스터(Mob) 풀링은 불필요하며, 도입 시 안정성과 개발 생산성을 저해합니다.
2. **기존 파이프라인 유지:** `AFCMobCampSpawner`의 네이티브 `SpawnActor` / `Destroy` 구조를 그대로 유지합니다.
3. **최적화 자원 재배치:** 풀링 엔지니어링 리소스는 발사체 풀링(``FCProjectileBase``) 및 이펙트/사운드 캐싱에 집중하는 것이 프로젝트 품질과 성능 관점에서 최적의 전략입니다.